

MODULE 16 · PART 5 - RESHAPING

Pivot and Unpivot

Rows into columns, columns into rows, and why only one of those is easy.

PART 5 of 8 MODULE 16 of 26 RUNS about 30 min

Why reshaping gets its own part

- ◆ **It is the only part about the shape of a result** rather than about which rows are in it or what they contain.
- ◆ **And the two directions are not symmetrical**, which is the surprise. One is trivial and one runs into a wall.
- ◆ **The wall is worth understanding once**, because it explains six engines' syntax and every workaround you will meet.
- ◆ **It is also the shortest part**, at two modules, because once you have the wall the rest is vocabulary.

THE TAKEAWAY

Every other part of this course is about which rows and what values. This one is about the arrangement, and arrangement is where SQL is least flexible.

A query's **column list is fixed before it runs.**
A pivot wants columns that depend on the data.

Long and wide

LONG, ONE ROW PER COMBINATION

region format net_sales

1 Convenience 4,723,189

1 Supermarket 10,727,169

1 Superstore 4,390,639

21 rows, 3 columns

WIDE, ONE COLUMN PER VALUE

region Convenience Supermarket Superstore

1 4,723,189 10,727,169 4,390,639

2 4,357,547 11,441,990 4,017,413

...

7 rows, 4 columns

THE TAKEAWAY

The same 21 numbers. Long is what a database wants to store and join; wide is what a person wants to read. Reshaping is the conversion between the two.

Reading a pivot out loud

WRITTEN

SAID

```
SUM(CASE WHEN f='X' THEN v END) AS x
```

a column called x, holding the total of v where f was X

```
PIVOT (SUM(v) FOR f IN ('X','Y'))
```

two columns, X and Y, and drop everything else

```
PIVOT ... FOR f IN (ANY)
```

read the data first, then decide how many columns there are

```
UNPIVOT (v FOR f IN (x, y))
```

fold columns x and y into a name column and a value column

```
crosstab('...') AS t(a INT, b NUMERIC)
```

pivot the rows, and here is the shape I promise you will get

THE TAKEAWAY

Row two is the one people misread. 'And drop everything else' is not in the syntax anywhere, and it is what the clause does.

Why long is the right shape to store

- ◆ **A new format is a new row**, not a schema change. Wide form needs a migration for every new value.
- ◆ **Filtering and joining work normally**, because the value is data rather than a column name.
- ◆ **And wide form cannot be aggregated across its columns** without naming every one of them.
- ◆ **So pivot last, at the edge**, where a person or a spreadsheet is about to read it.

THE TAKEAWAY

Every argument for long form is an argument about change. A wide table is correct until somebody adds a category, and then it is a schema ticket.

Conditional aggregation. **No keyword, no dialect, and it has always worked.**

Pivot, the portable way

```
1 SELECT region_key,  
2        SUM(CASE WHEN store_format='Convenience' THEN net_sales END)  
3          AS convenience,  
4        SUM(CASE WHEN store_format='Supermarket' THEN net_sales END)  
5          AS supermarket,  
6        SUM(CASE WHEN store_format='Superstore' THEN net_sales END)  
7          AS superstore  
8 FROM region_format  
9 GROUP BY region_key;
```

- 2 One aggregate per column, and the column names are written by you rather than derived from the data.
- 8 No ELSE, so non-matching rows are null and SUM skips them. Module seven's mechanism, doing the work.

THE TAKEAWAY

Three columns because you typed three. That is the limitation and the guarantee at the same time: the shape is in the SQL, where a reviewer can see it.

The same thing with FILTER

```
1 SELECT region_key,  
2        SUM(net_sales) FILTER (WHERE store_format='Convenience')  
3        AS convenience,  
4        SUM(net_sales) FILTER (WHERE store_format='Supermarket')  
5        AS supermarket,  
6        SUM(net_sales) FILTER (WHERE store_format='Superstore')  
7        AS superstore  
8 FROM region_format  
9 GROUP BY region_key;
```

2 Reads better, and runs on Postgres, SQLite and DuckDB only. Module seven's portability note, arriving again.

THE TAKEAWAY

Identical meaning, identical plan. FILTER says what it means and CASE runs everywhere, and that trade has not changed since module seven.

Most engines have a PIVOT clause now. **It still needs the column list.**

Every pivot **aggregates**. If the grid looked like it did not, the aggregate was hiding duplicates.

When the aggregate is hiding something

```
1 -- source: one row per region per format, you believe
2 region  format      net_sales
3       1  Supermarket 6,200,000
4       1  Supermarket 4,527,169  <- a second row nobody expected
5
6 PIVOT (SUM(net_sales) FOR store_format IN (...))
7
8 -- the grid shows 10,727,169 for region 1, Supermarket.
9 -- correct if the duplication was real, wrong if it was a bad join.
```

9 The pivot cannot tell you which. It combines whatever arrives, and the grid looks identical either way.

THE TAKEAWAY

Assert the source grain before pivoting: row count equals distinct count of the grid keys. A pivot is the last place a duplicate becomes invisible.

PIVOT, with the list

```
1 -- Redshift, Snowflake, BigQuery, Oracle
2 SELECT * FROM region_format
3 PIVOT (SUM(net_sales)
4       FOR store_format IN ('Convenience','Supermarket','Superstore')));
5
6 -- SQL Server: same idea, square brackets, and an alias is required
7 SELECT * FROM region_format
8 PIVOT (SUM(net_sales)
9       FOR store_format IN ([Convenience],[Supermarket],[Superstore])) p;
```

4 The IN list is the column list. It is not optional, and it is the whole subject of this module.

THE TAKEAWAY

Four lines instead of nine, and the same three values written out by hand. The keyword is shorter and it knows exactly as much as you told it.

That column list is **also a filter**. Values you did not list are silently excluded.

What a missing value does

```
1 -- the data now contains four formats
2 Convenience Supermarket Superstore Franchise
3
4 -- the pivot still lists three
5 PIVOT (SUM(net_sales)
6       FOR store_format IN ('Convenience','Supermarket','Superstore'))
7
8 -- Franchise sales: not a column, not an error, not in the total
9 -- the report still renders and still reconciles against itself
```

8 The report is internally consistent and no longer describes the business. Nothing in it says a format is missing.

THE TAKEAWAY

This is module seven's filtered denominator, in a different clause. The output is correct for the population it covers, and the population changed without telling anybody.

A weekly regional pack pivots net sales by store format, with the three formats listed explicitly. It has run unchanged for two years and reconciles to the penny against itself every week.

Kestrel opens 40 franchise sites in March, a fourth format. In June, finance notice the pack's grand total is about 3% below the group figure. The pack has not changed and produces no errors.

What happened?

- ☐ A The franchise sites have null sales in the source, so they contribute nothing to any aggregate.
- ☐ B The pivot needs a GROUP BY on the new format column before it will include it.
- ☐ C The engine has a column limit and silently truncated the result at three pivoted columns.
- ☐ D The IN list is a filter as well as a shape, so a format that is not listed is excluded entirely.

◆ PAUSE AND DECIDE

What happened?

A Null sales at source

Would show the format as a column of nulls if it were listed. It is not listed, so the column does not exist at all.

C A column limit

Limits are in the thousands and they raise errors. Three columns is nowhere near any engine's limit.

B A missing GROUP BY

PIVOT derives its grouping from the columns not mentioned in the clause. Adding a GROUP BY is not how the new value gets in.

D The list is a filter

Unlisted values are excluded with no error. The pack is correct for three formats and the business now has four.

THE SIGNAL

"Reconciles to the penny against itself." Internal consistency is exactly what this defect preserves. Only a comparison against something outside the pack could find it.

Guarding a pivot

- ◆ **Assert the pivoted total against the unpivoted total.** One comparison, and it catches every missing value.
- ◆ **Or add an 'other' column,** with a NOT IN on the listed values, so anything unexpected is visible rather than gone.
- ◆ **Or assert the distinct count of the pivot column** against the length of your list, which fails the moment a value appears.
- ◆ **Reconciling the pivot against itself proves nothing,** because the excluded rows are absent from both sides.

THE TAKEAWAY

The second option is the one worth adopting as a habit. An 'other' column that is always zero is a test that runs on every execution and costs one line.

The 'other' column

```
1 SELECT region_key,  
2        SUM(CASE WHEN store_format='Convenience' THEN net_sales END) AS convenience,  
3        SUM(CASE WHEN store_format='Supermarket' THEN net_sales END) AS supermarket,  
4        SUM(CASE WHEN store_format='Superstore' THEN net_sales END) AS superstore,  
5        SUM(CASE WHEN store_format NOT IN ('Convenience', 'Supermarket',  
6        'Superstore') THEN net_sales END) AS other  
7 FROM region_format  
8 GROUP BY region_key;
```

6 Always zero, until the day it is not. A column that exists to be empty is a test that runs every time.

THE TAKEAWAY

It also handles nulls, which NOT IN does not. A null store_format satisfies neither the listed values nor the NOT IN, so it vanishes from both.

Can the columns come **from the data**?
Yes, and it costs two round trips, always.

Months as columns

```
1 SELECT store_key,  
2        SUM(net_sales) FILTER (WHERE retail_month = 1) AS m01,  
3        SUM(net_sales) FILTER (WHERE retail_month = 2) AS m02,  
4        -- ... ten more ...  
5        SUM(net_sales) FILTER (WHERE retail_month = 12) AS m12,  
6        SUM(net_sales) AS full_year  
7 FROM agg_store_month  
8 GROUP BY store_key;
```

6 The row total, computed from the source rather than by adding twelve columns. It is correct even if a month is missing.

THE TAKEAWAY

The commonest pivot there is, and the only one where the column list genuinely is stable. Twelve months is a fact about calendars, not about your data.

Row totals and column totals

- ◆ **Compute row totals from the source**, not by adding the pivoted columns. A null column poisons the addition.
- ◆ **Adding twelve nullable columns gives null** whenever any one of them is null, which is module two in a grid.
- ◆ **Or coalesce each column to zero before adding**, which is correct and asserts that a missing month means no sales.
- ◆ **Column totals are a different query**, or a GROUPING SETS, which is module nineteen.

THE TAKEAWAY

A row of a pivoted grid is a row of columns, and SQL has almost nothing that operates across columns. Every across-the-row calculation is written out by hand.

Percent of row, after pivoting

```
1 SELECT store_key,  
2         m01, m02, m12,  
3         m01 / NULLIF(full_year, 0) AS m01_share,  
4         m02 / NULLIF(full_year, 0) AS m02_share  
5 FROM pivoted;
```

3 One expression per column, written out. There is no way to say 'divide every month column by the total'.

THE TAKEAWAY

This is what 'a value became a name' costs. In long form it was one division. In wide form it is twelve, and thirteen when a thirteenth period appears.

A pivot turns **a value into a name**. Names are query text, and nothing in SQL computes on them.

Who does the two round trips

APPROACH	WHERE THE DISCOVERY HAPPENS
Hand-written	You query the distinct values, then paste them into the pivot.
A templating layer	The transformation tool queries, generates the SQL, then runs it.
Snowflake PIVOT ... IN (ANY)	The engine does it internally, and it is still two passes.
DuckDB PIVOT	Same. It infers the column list by reading the pivot column first.
PostgreSQL crosstab()	It does not. You must supply a column definition list regardless.

THE TAKEAWAY

The engines that look dynamic are doing the same two steps you would, with the seam hidden. That is a real convenience and it is not a different mechanism.

What dynamic actually looks like

```
1 -- DuckDB: the column list is inferred
2 PIVOT region_format ON store_format USING SUM(net_sales)
3   GROUP BY region_key;
4 -- returns: region_key, Convenience, Supermarket, Superstore
5
6 -- Snowflake: ANY, optionally ordered
7 SELECT * FROM region_format
8 PIVOT (SUM(net_sales) FOR store_format IN (ANY ORDER BY store_format));
9
10 -- Postgres: still a column definition list, even with tablefunc
11 SELECT * FROM crosstab('...') AS t(region INT, conv NUMERIC, ...);
```

- 8 ANY reads the values first. Convenient, and it means the result shape can change between runs without warning.
- 11 The extension pivots the rows and still cannot name the columns for you.

THE TAKEAWAY

Verified on the capstone database: DuckDB's PIVOT on a 1,500-value column produced 1,501 columns without complaint, and without being asked whether that was wise.

What dynamic costs you

- ◆ **The result shape changes between runs.** A new value appears and every downstream consumer gets a column it has never seen.
- ◆ **Positional consumers break.** Anything reading by index rather than by name shifts, exactly as in module eight's leak.
- ◆ **And a view cannot have a changing shape,** so a dynamic pivot cannot be wrapped in one and reused.
- ◆ **So dynamic is right at the very edge,** feeding a person or an export, and wrong anywhere something reads it programmatically.

THE TAKEAWAY

The explicit list is a filter that hides data. The dynamic form is a contract that changes underneath consumers. Neither is safe, and they are unsafe in opposite directions.

A team on Redshift wants a pivot whose columns follow the data, so that a new store format appears automatically. The output is a table that four downstream transformation jobs read by column name.

An engineer has found that Snowflake supports PIVOT with ANY and asks whether Redshift has an equivalent, or whether a recursive CTE could generate the columns.

What should they be told?

- ☐ A A pivot's shape must be known before it runs, so any dynamic pivot is two round trips. Generate the SQL, and keep the explicit list in the generated code.
- ☐ B Redshift's PIVOT accepts ANY in the IN list, in the same way Snowflake's does.
- ☐ C A recursive CTE can produce a column per value, because recursion adds rows and columns as it goes.
- ☐ D Return the result in long form and let each downstream job pivot it for itself.

◆ PAUSE AND DECIDE

What should they be told?

- A

 Two round trips, generated
Discovery then execution, with the list visible in the generated SQL. Four consumers then see a named, reviewable contract rather than a shifting one.
- B

 Redshift accepts ANY
It does not. Snowflake and DuckDB infer the list; Redshift, BigQuery, Oracle and SQL Server all require it written out.
- C

 A recursive CTE
Recursion adds rows. Every pass must produce the same columns as the anchor, so it cannot widen a result by even one column.
- D

 Keep it long
Often the right answer, and not here. Four jobs pivoting for themselves is four column lists that will disagree within a year.

THE SIGNAL

"Four downstream jobs read it by column name." That sentence rules out a shape that changes, which is exactly what the dynamic form provides.

Unpivot is **far easier**, because the output shape is fixed no matter what the input holds.

Unpivot, two ways

```
1 -- the keyword: Redshift, Snowflake, BigQuery, Oracle, SQL Server, DuckDB
2 SELECT * FROM wide
3 UNPIVOT (net_sales FOR store_format IN (convenience, supermarket, superstore));
4
5 -- the portable way: a UNION ALL per column
6 SELECT region_key, 'Convenience' AS store_format, convenience AS net_sales FROM wide
7 UNION ALL SELECT region_key, 'Supermarket', supermarket FROM wide
8 UNION ALL SELECT region_key, 'Superstore', superstore FROM wide;
```

6 One SELECT per column, unioned. It reads every row once per column, which is the cost of the portable form.

THE TAKEAWAY

The output is always three columns, whichever form you use. That is why unpivot has no dynamic problem: the shape was never in question.

The asymmetry, measured

```
1 long form                21 rows
2   -> PIVOT               7 rows x 3 value columns = 21 cells
3   -> set 2 cells to NULL
4   -> UNPIVOT              19 rows
5
6 -- UNPIVOT excludes NULL cells by default.
7 -- The round trip is lossy exactly where the grid is sparse.
```

4 Measured on the capstone database. Two null cells, two rows fewer, and no warning.

THE TAKEAWAY

A pivot manufactures nulls for combinations that did not exist. Unpivot then removes them, which restores the original sparse long form, and also removes any null you meant.

Round-tripping, and when it is lossy

- ◆ **Long to wide to long is lossless**, when every null in the grid was manufactured by the pivot itself.
- ◆ **And lossy when a null meant something**, because unpivot cannot tell a missing combination from a recorded unknown.
- ◆ **INCLUDE NULLS where the engine offers it**, which keeps every cell and makes the loss a decision rather than a default.
- ◆ **Zero and null are different here too**. A coalesce before the pivot turns a sparse grid into a dense one and changes the row count.

THE TAKEAWAY

The pivot created nulls and the unpivot destroyed them, and the two happened to cancel. That is not the same as the round trip being safe.

Unpivoting a wide source

```
1 -- a source system delivers one column per month
2 SELECT store_key, jan, feb, mar, apr, may, jun,
3        jul, aug, sep, oct, nov, dec FROM legacy_extract;
4
5 SELECT store_key, month_name, net_sales
6        FROM legacy_extract
7 UNPIVOT (net_sales FOR month_name IN
8         (jan, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec));
```

8 Twelve columns named once, and the output is three columns regardless. This is the direction that scales.

THE TAKEAWAY

The most common real use of UNPIVOT: normalising a wide extract from a system that thinks in spreadsheets. It is the first step of every such load.

Unpivoting more than one measure

- ◆ **Two measures per period means two value columns**, so sales and units for twelve months is twenty four input columns.
- ◆ **Most engines allow a multi-column UNPIVOT**, folding pairs together into two value columns and one name column.
- ◆ **Where they do not, it is two UNPIVOTs and a join**, on the key and the name, which is ordinary and tedious.
- ◆ **And the column naming convention becomes load-bearing**, because the join depends on parsing sales_m01 and units_m01 into a shared key.

THE TAKEAWAY

A wide extract with two measures encodes structure in its column names. Unpivoting it means parsing those names, which is string manipulation standing in for a schema.

Pivot and GROUPING SETS

- ◆ **A pivot gives you the grid.** GROUPING SETS gives you the subtotals and the grand total in the same pass.
- ◆ **They compose.** Group with GROUPING SETS first, then pivot the result, and the subtotal rows pivot along with everything else.
- ◆ **But subtotal rows carry nulls in the grouping columns,** which a pivot will treat as a value or drop, depending on your list.
- ◆ **So flag the subtotals before pivoting,** with GROUPING(), which module nineteen covers properly.

THE TAKEAWAY

The capstone's d08 is exactly this composition, and its harness checks that a subtotal row is distinguishable from a detail row. That flag is why.

A pipeline pivots a long-form measure table to wide for a spreadsheet export, and a second job unpivots the exported file back to long to load it into a downstream model.

The round trip loses rows. The long table has 84,120 rows and the reloaded table has 71,338. Every row that survives has correct values, and the pivot and unpivot both name the same columns.

What is the cause?

- ☐ A The pivot aggregated away duplicate combinations, so the long form had more rows than the grid has cells.
- ☐ B The column order changed in the export, so the unpivot mapped values to the wrong names and some were rejected.
- ☐ C UNPIVOT excludes NULL cells by default, and the pivot created a NULL for every combination that did not exist.
- ☐ D The value column type is narrower after the export, so some values failed to convert and were dropped.

◆ PAUSE AND DECIDE

What is the cause?

A Duplicates aggregated away

A real hazard, and it would mean the long form was not at the grain it claimed. It also loses rows at the pivot, not at the unpivot.

C NULL cells excluded

The grid is sparse, so the pivot manufactured nulls for absent combinations and the unpivot removed them again.

B Column order changed

Would produce wrong values rather than missing rows, and the scenario says every surviving row is correct.

D A type conversion

Would fail loudly on most loaders, and it would not correlate with which combinations happened to exist.

THE SIGNAL

"Every row that survives has correct values." A loss with no corruption points at rows being filtered rather than mangled, and NULL is what gets filtered here.

Should this be pivoted **in SQL at all?**

When to pivot in SQL

- ◆ **When the wide form is the artefact.** A file somebody opens, a fixed-format export a partner ingests.
- ◆ **When the column count is small and stable**, and stable means somebody owns the list and reviews it.
- ◆ **Not to feed a BI tool**, which pivots better, dynamically, and without a schema change when a value appears.
- ◆ **And not as an intermediate step**, because everything after a pivot is harder: no filtering, no joining, no aggregating across.

THE TAKEAWAY

A pivoted intermediate is the worst of both. You have paid for the reshape and lost the ability to do anything else with the result.

What you lose by pivoting early

- ◆ **Filtering.** 'Only supermarkets' was a WHERE clause; now it is a choice of which column to select.
- ◆ **Joining.** The format was a key you could join on; now it is a column name, which you cannot.
- ◆ **Aggregating across.** A total needs every column named, and a new one silently stays out of it.
- ◆ **And adding a measure multiplies the columns,** because two measures across three formats is six columns, not four.

THE TAKEAWAY

Every one of these is the same loss: a value became a name. Names are not data, and nothing in SQL operates on them.

Two measures, six columns

```
1 -- DuckDB, pivoting two aggregates at once
2 PIVOT region_format ON store_format
3   USING SUM(net_sales), SUM(lines)
4   GROUP BY region_key;
5
6 -- columns produced:
7 --   region_key
8 --   Convenience_sum(net_sales)   Convenience_sum(lines)
9 --   Supermarket_sum(net_sales)   Supermarket_sum(lines)
10 --   Superstore_sum(net_sales)   Superstore_sum(lines)
```

- 8 The engine names them by cross-producting the value with the aggregate. Three formats times two measures is six columns.

THE TAKEAWAY

Measured on the capstone database. A third measure makes nine columns, and a fourth format makes twelve. The width is a product, not a sum.

An analyst pivots sales by product category to give merchandising a grid. There are 1,500 categories. The query runs, produces 1,501 columns, and the engine raises no error.

The result cannot be opened in the team's spreadsheet tool, and the downstream model that was meant to read it fails on column count. The analyst asks how to make the tooling handle it.

What should they be told?

- ☐ A Increase the cluster size and the client's column limit, since the query itself is correct and completes.
- ☐ B 1,501 columns is not a table anybody can use. Pivot the top N categories, group the rest as 'other', or keep it long.
- ☐ C Unpivot the result afterwards to bring it back to a manageable shape before handing it over.
- ☐ D Split the output into fifteen tables of a hundred columns each, so that each one is within the tooling's limits.

◆ PAUSE AND DECIDE

What should they be told?

A Raise the limits

The query completes already; the engine is not the constraint. It makes an unusable artefact available in more places.

C Unpivot it back

Returns it to the long form it started as, having paid for the pivot. Correct destination, absurd route.

B It is not a usable table

Nobody reads 1,500 columns. Top 20 plus an 'other' column answers the actual question and fits on a screen.

D Split into fifteen tables

Fifteen unusable tables instead of one. It also breaks any comparison across categories, which is the point of a grid.

THE SIGNAL

"The engine raises no error." Producing 1,501 columns is a capability, not an endorsement. The question was never whether it could run.

Testing a pivot

- ◆ **Source grain first.** Row count equals distinct count of the grid keys, before any reshaping happens.
- ◆ **Then the total.** Sum of every pivoted column equals the long-form total, which catches an unlisted value.
- ◆ **Then the row count.** One row per distinct key, which catches a grid key you forgot to group by.
- ◆ **And a fixture with a value not in your list,** which is the only way to see the silent exclusion happen.

THE TAKEAWAY

Four assertions, and the fourth is the one nobody writes. A pivot tested only on data whose values you listed cannot fail the way scenario one failed.

The pivot assertion

```
1 SELECT
2   (SELECT ROUND(SUM(net_sales),2) FROM region_format)      AS long_total,
3   (SELECT ROUND(SUM(COALESCE(convenience,0)
4     + COALESCE(supermarket,0)
5     + COALESCE(superstore,0)
6     + COALESCE(other,0)),2) FROM pivoted)      AS wide_total,
7   (SELECT COUNT(DISTINCT store_format) FROM region_format) AS values_present,
8   (SELECT COUNT(*) FROM pivoted WHERE other <> 0)      AS unlisted_rows;
```

- 6 Coalesce each column, or one null makes the whole sum null and the assertion passes by accident.
- 8 And the 'other' column doing its job: non-zero means a value appeared that nobody listed.

THE TAKEAWAY

Two totals that must match, a count you can eyeball, and a column that must be zero. Four numbers, and they cover every failure in this module.

In a transformation tool

- ◆ **Most tools ship a pivot macro**, which queries the distinct values at compile time and writes the column list for you.
- ◆ **The generated SQL still contains an explicit list**, which means it is reviewable, diffable, and stable between runs.
- ◆ **And the list changes when the data does**, so a new value silently changes your model's schema at the next build.
- ◆ **Pin the list where the contract matters**. Generated is right for exploration, pinned is right for anything downstream reads.

THE TAKEAWAY

A compile-time pivot is the two round trips again, moved to build time. That is the best place for them, and it still means the schema depends on the data.

The generated pivot

```
1 -- what you write
2 {{ pivot('store_format', values=dbt_utils.get_column_values(
3         ref('region_format'), 'store_format'),
4         agg='sum', then_value='net_sales') }}
5
6 -- what gets compiled and run
7 sum(case when store_format = 'Convenience' then net_sales end)
8   as "Convenience",
9 sum(case when store_format = 'Supermarket' then net_sales end)
10  as "Supermarket", ...
```

7 Conditional aggregation, which is where we started. Every pivot compiles to this, whoever wrote it.

THE TAKEAWAY

The macro runs a query at compile time to get the values, then writes the CASE expressions. It is the two round trips, made explicit and put in version control.

Deciding, in four questions

01

Why

Is the wide form the artefact, or the presentation? If presentation, do not pivot in SQL.

02

How many

How many columns, and who owns the list? More than about twenty and it is not a grid.

03

New values

What happens when a new value appears? Silently excluded, or a shifting shape. Pick deliberately.

04

Assert

Pivoted total against unpivoted total, every run. One comparison, and it catches the silent exclusion.

THE TAKEAWAY

Question three has no good answer, only two bad ones. Choosing between them consciously is the whole of good pivot practice.

The confusions, settled

BELIEF

PIVOT saves me writing the column list

The IN list only controls the output shape

A dynamic pivot is one statement

Pivot and unpivot are inverses

Postgres can pivot with tablefunc

A recursive CTE could generate columns

THE TRUTH

It does not. Only DuckDB and Snowflake infer it.

It filters as well. Unlisted values are dropped silently.

It is two passes, wherever the seam is hidden.

Not when the grid is sparse. UNPIVOT drops NULL cells.

It can, and you still declare every output column.

No construct in SQL can widen a result at runtime.

THE TAKEAWAY

Every row here is the same fact restated: the column list is part of the query text, not part of the data. Nothing in SQL moves it across that line.

Five habits

- ◆ **Pivot last, at the edge**, where nothing downstream needs to filter, join or aggregate across the result.
- ◆ **Add an 'other' column**, which should be zero forever and which tells you the day a new value appears.
- ◆ **Assert the pivoted total against the long-form total**, because the pivot reconciling against itself proves nothing.
- ◆ **Use INCLUDE NULLS when unpivoting**, or accept the loss knowingly rather than discovering it in a row count.
- ◆ **And prefer conditional aggregation** when the code must be portable, because it has always worked and always will.

THE TAKEAWAY

The second and third are one line each and between them they catch every defect in this module. Neither is default behaviour anywhere.

Long form is **a database's shape**. Wide form is **a page's shape**. Convert at the boundary.

Where the boundary usually is

- ◆ **The reporting tool.** It pivots dynamically, without a column list, and it is what the tool is for.
- ◆ **The export.** A file a person opens, where the wide shape is genuinely the deliverable.
- ◆ **Not the model.** A pivoted model cannot be filtered, joined or extended without a schema change.
- ◆ **And not the intermediate,** ever. Pivoting midway means every step after it works with names instead of values.

THE TAKEAWAY

The rule that follows: if anything downstream is SQL, do not pivot. If the next consumer is a person or a file, pivot.

What Part 5 is really about

- ◆ **This module: values becoming names**, which is one-way, and which the whole module is a consequence of.
- ◆ **Module seventeen: rows becoming rows**. One row into many, and many back into one.
- ◆ **Those are much easier**, because the column list never changes. Only the row count does.
- ◆ **Which is the theme**. SQL is relaxed about how many rows a query returns and rigid about how many columns.

THE TAKEAWAY

Row count is decided at runtime and column count at parse time. That single asymmetry explains this module, the next one, and why one is short.

Pivoting made columns from values.
Module seventeen makes rows from one row.